

Database Design

Managed tutoring marketplace — PostgreSQL schema, v2.0

Revision note: rewritten against production-grade conventions — surrogate/public key split, integer money, explicit soft-delete policy, full DDL with constraints and indexes, security/compliance and scaling notes. Supersedes v1.0.

1. Conventions & design principles

1.1 Naming

- Tables: plural, snake_case (learner_profiles, teacher_availability).
- Columns: snake_case; boolean columns prefixed is_/has_; timestamps suffixed _at; foreign keys named <singular_table>_id.
- Primary keys are always literally named id; do not embed the table name in the PK column.

1.2 Key strategy: surrogate BIGINT + public UUID

Every table uses a BIGINT identity column as the physical primary key (fast joins, small index footprint, sequential inserts). A separate public_id UUID column is exposed to APIs, URLs, and clients. This avoids enumeration/scraping of internal ids (a parent's child roster, a teacher's session count) while keeping join performance on the internal key. Never expose id externally.

```
id          BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,  
public_id   UUID NOT NULL DEFAULT gen_random_uuid() UNIQUE,
```

1.3 Timestamps & audit columns

- All timestamps are TIMESTAMPTZ, stored in UTC; convert at the presentation layer, never in the database.
- created_at defaults to now(); updated_at is maintained by a single shared trigger, not application code (prevents drift when rows are edited via scripts/migrations).

```
CREATE OR REPLACE FUNCTION set_updated_at() RETURNS TRIGGER AS $$  
BEGIN NEW.updated_at = now(); RETURN NEW; END;  
$$ LANGUAGE plpgsql;  
-- attached per-table as: CREATE TRIGGER trg_<table>_updated_at  
-- BEFORE UPDATE ON <table> FOR EACH ROW EXECUTE FUNCTION set_updated_at();
```

1.4 Soft delete: applied selectively, not globally

deleted_at TIMESTAMPTZ NULL is added only to entities a person can reasonably “remove” and where recovery matters: accounts, learner_profiles, teachers. Transactional and evidentiary tables — sessions, payments, payouts, feedback, ratings — are never soft-deleted; they are append-only and move through status values instead. Financial and academic records must remain queryable for dispute resolution and audit regardless of what a user does to their profile.

1.5 Money: integer minor units, never FLOAT/DECIMAL-as-display

All monetary amounts are stored as BIGINT in the currency's smallest unit (e.g. kobo for NGN), paired with a currency_code CHAR(3). This eliminates floating-point rounding entirely; formatting to major units (/100) happens only at display time.

```
amount_minor  BIGINT NOT NULL CHECK (amount_minor >= 0),  
currency_code CHAR(3) NOT NULL DEFAULT 'NGN',
```

1.6 Enumerated values: lookup tables for open domains, CHECK constraints for closed ones

Native Postgres ENUM types are avoided — adding a value later requires a blocking ALTER TYPE in older Postgres versions and complicates zero-downtime deploys. The rule applied here:

- Domains that will genuinely grow over time (subjects, curricula, payment methods) are lookup tables with a surrogate key — new rows, not schema migrations, add a new subject.
- Domains that are small, stable, and part of core business logic (session status, payment status, verification status) are a VARCHAR with a CHECK constraint — a lookup table for a 4-value status most callers will hardcode in application logic anyway adds a join for no real flexibility gain.

1.7 Referential integrity policy

- ON DELETE RESTRICT (default posture) on any FK pointing at a financial or academic record — you cannot delete a teacher or learner profile out from under existing sessions/payments; deactivate (deleted_at) instead.
- ON DELETE CASCADE is used narrowly, only for true ownership compositions with no independent meaning: teacher_availability rows, teacher_subjects/teacher_curricula join rows, and assessment_subjects join rows die with their parent.
- ON DELETE SET NULL is used where the child record should survive its parent's removal as a historical fact: learner_profiles.linked_login_account_id, sessions.match_id.

1.8 Indexing philosophy

- Every foreign key column is indexed explicitly — Postgres does not do this automatically.
- Composite indexes are built for the actual hot read paths (a learner's upcoming sessions, a teacher's pending matches), not speculatively for every column combination.
- Partial indexes are used for status-filtered queries that only ever touch a small slice of a large table (e.g. sessions WHERE status = 'scheduled').

2. Reference / lookup tables

```
CREATE TABLE subjects (  
  id          BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,  
  code        VARCHAR(40) NOT NULL UNIQUE,          -- 'mathematics', 'further_maths'  
  display_name VARCHAR(80) NOT NULL,  
  is_active   BOOLEAN NOT NULL DEFAULT true  
);  
  
CREATE TABLE curricula (  
  id          BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,  
  code        VARCHAR(40) NOT NULL UNIQUE,          -- 'ng_waec_neco', 'uk_igcse', 'us_common_core', 'ib'  
  display_name VARCHAR(80) NOT NULL,  
  is_active   BOOLEAN NOT NULL DEFAULT true  
);  
  
CREATE TABLE payment_methods (  
  id          BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,  
  code        VARCHAR(40) NOT NULL UNIQUE,          -- 'card', 'bank_transfer', 'usd', 'mobile_money'  
  display_name VARCHAR(80) NOT NULL,  
  is_active   BOOLEAN NOT NULL DEFAULT true  
);
```

3. Core schema

3.1 Identity & access

```
CREATE TABLE accounts (  
  id          BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,  
  public_id   UUID NOT NULL DEFAULT gen_random_uuid() UNIQUE,  
  email       CITEXT NOT NULL UNIQUE,              -- case-insensitive  
  phone       VARCHAR(20),  
  password_hash VARCHAR(255) NOT NULL,             -- argon2id  
  account_type VARCHAR(20) NOT NULL  
              CHECK (account_type IN ('parent', 'independent_student', 'teacher')),  
  email_verified_at TIMESTAMPTZ,  
  created_at   TIMESTAMPTZ NOT NULL DEFAULT now(),  
  updated_at   TIMESTAMPTZ NOT NULL DEFAULT now(),  
  deleted_at   TIMESTAMPTZ
```

```
);
CREATE UNIQUE INDEX ux_accounts_email ON accounts (email) WHERE deleted_at IS NULL;
CREATE INDEX ix_accounts_type ON accounts (account_type) WHERE deleted_at IS NULL;
```

3.2 Learner & assessment

```
CREATE TABLE learner_profiles (
  id BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  public_id UUID NOT NULL DEFAULT gen_random_uuid() UNIQUE,
  owner_account_id BIGINT NOT NULL REFERENCES accounts(id) ON DELETE RESTRICT,
  linked_login_account_id BIGINT REFERENCES accounts(id) ON DELETE SET NULL,
  profile_type VARCHAR(10) NOT NULL CHECK (profile_type IN ('child','own')),
  full_name VARCHAR(120) NOT NULL,
  date_of_birth DATE,
  grade_level VARCHAR(60) NOT NULL,
  curriculum_id BIGINT NOT NULL REFERENCES curricula(id) ON DELETE RESTRICT,
  created_at TIMESTAMPTZ NOT NULL DEFAULT now(),
  updated_at TIMESTAMPTZ NOT NULL DEFAULT now(),
  deleted_at TIMESTAMPTZ,
  CONSTRAINT chk_child_has_no_self_login
    CHECK (NOT (profile_type = 'own' AND linked_login_account_id IS NOT NULL))
);
CREATE INDEX ix_learner_profiles_owner ON learner_profiles (owner_account_id) WHERE deleted_at IS NULL;
CREATE INDEX ix_learner_profiles_linked_login ON learner_profiles (linked_login_account_id)
  WHERE linked_login_account_id IS NOT NULL;

CREATE TABLE assessments (
  id BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  learner_profile_id BIGINT NOT NULL REFERENCES learner_profiles(id) ON DELETE RESTRICT,
  academic_challenges TEXT[],
  learning_goals TEXT[] NOT NULL,
  budget_tier VARCHAR(10) NOT NULL CHECK (budget_tier IN ('basic','standard','premium')),
  preferred_format VARCHAR(15) NOT NULL
    CHECK (preferred_format IN ('one_on_one','group','no_preference')),
  session_frequency VARCHAR(15) NOT NULL
    CHECK (session_frequency IN ('weekly','twice_weekly','custom')),
  availability JSONB NOT NULL, -- [{day, start, end}, ...] – no fixed cardinality, JSONB is
appropriate
  consent_given BOOLEAN NOT NULL DEFAULT false,
  submitted_at TIMESTAMPTZ NOT NULL DEFAULT now(),
  CONSTRAINT chk_minor_requires_consent CHECK (
    consent_given = true OR NOT EXISTS (
      SELECT 1 FROM learner_profiles lp
      WHERE lp.id = learner_profile_id AND lp.profile_type = 'child'
    )
  )
);
CREATE INDEX ix_assessments_learner ON assessments (learner_profile_id, submitted_at DESC);

CREATE TABLE assessment_subjects ( -- many-to-many: an assessment can name several subjects
  assessment_id BIGINT NOT NULL REFERENCES assessments(id) ON DELETE CASCADE,
  subject_id BIGINT NOT NULL REFERENCES subjects(id) ON DELETE RESTRICT,
  PRIMARY KEY (assessment_id, subject_id)
);
```

Note: the true-SQL check constraint on consent (chk_minor_requires_consent) illustrates intent but a subquery inside a CHECK is not enforced by Postgres across concurrent transactions — in production this rule belongs in a BEFORE INSERT trigger or the application's write path, not a CHECK. Flagged here rather than silently “fixed” so the trade-off is visible.

3.3 Teacher & verification

```
CREATE TABLE teachers (
  id                BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  public_id         UUID NOT NULL DEFAULT gen_random_uuid() UNIQUE,
  account_id        BIGINT NOT NULL UNIQUE REFERENCES accounts(id) ON DELETE RESTRICT,
  years_experience   SMALLINT NOT NULL CHECK (years_experience >= 0),
  bio               TEXT,
  preferred_format   VARCHAR(15) NOT NULL
                      CHECK (preferred_format IN ('one_on_one', 'group', 'both')),
  rate_minor        BIGINT NOT NULL CHECK (rate_minor >= 0),
  currency_code      CHAR(3) NOT NULL DEFAULT 'NGN',
  id_verified_at    TIMESTAMPTZ,
  credentials_verified_at TIMESTAMPTZ,
  demo_status        VARCHAR(15) NOT NULL DEFAULT 'not_submitted'
                      CHECK (demo_status IN ('not_submitted', 'submitted', 'scheduled', 'completed')),
  verification_status VARCHAR(10) NOT NULL DEFAULT 'pending'
                      CHECK (verification_status IN ('pending', 'approved', 'rejected')),
  rating_avg         NUMERIC(3,2),          -- denormalized rollup, see section 6
  created_at         TIMESTAMPTZ NOT NULL DEFAULT now(),
  updated_at         TIMESTAMPTZ NOT NULL DEFAULT now(),
  deleted_at         TIMESTAMPTZ
);

CREATE INDEX ix_teachers_verification_status ON teachers (verification_status)
  WHERE deleted_at IS NULL;

CREATE TABLE teacher_subjects (
  teacher_id BIGINT NOT NULL REFERENCES teachers(id) ON DELETE CASCADE,
  subject_id BIGINT NOT NULL REFERENCES subjects(id) ON DELETE RESTRICT,
  PRIMARY KEY (teacher_id, subject_id)
);

CREATE TABLE teacher_curricula (
  teacher_id BIGINT NOT NULL REFERENCES teachers(id) ON DELETE CASCADE,
  curriculum_id BIGINT NOT NULL REFERENCES curricula(id) ON DELETE RESTRICT,
  PRIMARY KEY (teacher_id, curriculum_id)
);

CREATE TABLE teacher_availability (
  id                BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  teacher_id        BIGINT NOT NULL REFERENCES teachers(id) ON DELETE CASCADE,
  day_of_week       SMALLINT NOT NULL CHECK (day_of_week BETWEEN 0 AND 6),
  start_time        TIME NOT NULL,
  end_time          TIME NOT NULL,
  CONSTRAINT chk_availability_window CHECK (end_time > start_time)
);

CREATE INDEX ix_teacher_availability_teacher ON teacher_availability (teacher_id, day_of_week);
```

3.4 Matching & scheduling

```
CREATE TABLE matches (
  id                BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  learner_profile_id BIGINT NOT NULL REFERENCES learner_profiles(id) ON DELETE RESTRICT,
```

```

teacher_id          BIGINT NOT NULL REFERENCES teachers(id) ON DELETE RESTRICT,
match_reasoning     TEXT,
status              VARCHAR(10) NOT NULL DEFAULT 'proposed'
                    CHECK (status IN ('proposed','accepted','declined')),
decline_reason       TEXT,
created_at          TIMESTAMPTZ NOT NULL DEFAULT now()
);
CREATE INDEX ix_matches_learner_status ON matches (learner_profile_id, status);
CREATE INDEX ix_matches_teacher_status_pending ON matches (teacher_id)
  WHERE status = 'proposed';           -- partial index: teacher's pending-request inbox

CREATE TABLE sessions (
  id                 BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  public_id          UUID NOT NULL DEFAULT gen_random_uuid() UNIQUE,
  match_id           BIGINT REFERENCES matches(id) ON DELETE SET NULL,
  learner_profile_id BIGINT NOT NULL REFERENCES learner_profiles(id) ON DELETE RESTRICT,
  teacher_id         BIGINT NOT NULL REFERENCES teachers(id) ON DELETE RESTRICT,
  scheduled_at       TIMESTAMPTZ NOT NULL,
  format             VARCHAR(10) NOT NULL CHECK (format IN ('one_on_one','group')),
  is_trial           BOOLEAN NOT NULL DEFAULT false,
  status             VARCHAR(15) NOT NULL DEFAULT 'scheduled'
                    CHECK (status IN ('scheduled','in_progress','completed','no_show','cancelled')),
  created_at         TIMESTAMPTZ NOT NULL DEFAULT now()
);
CREATE INDEX ix_sessions_learner_scheduled ON sessions (learner_profile_id, scheduled_at DESC);
CREATE INDEX ix_sessions_teacher_scheduled ON sessions (teacher_id, scheduled_at DESC);
CREATE INDEX ix_sessions_upcoming ON sessions (scheduled_at)
  WHERE status = 'scheduled';         -- partial index: reminder/notification job

```

3.5 Payments & payouts

```

CREATE TABLE payments (
  id                 BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  public_id          UUID NOT NULL DEFAULT gen_random_uuid() UNIQUE,
  payer_account_id   BIGINT NOT NULL REFERENCES accounts(id) ON DELETE RESTRICT,
  session_id         BIGINT REFERENCES sessions(id) ON DELETE RESTRICT,  -- null for plan payments
  payment_method_id  BIGINT NOT NULL REFERENCES payment_methods(id) ON DELETE RESTRICT,
  plan_type          VARCHAR(15) NOT NULL
                    CHECK (plan_type IN ('per_session','weekly','monthly')),
  sessions_covered   SMALLINT NOT NULL CHECK (sessions_covered > 0),
  amount_minor       BIGINT NOT NULL CHECK (amount_minor >= 0),
  currency_code      CHAR(3) NOT NULL DEFAULT 'NGN',
  status             VARCHAR(12) NOT NULL DEFAULT 'processing'
                    CHECK (status IN ('processing','success','failed','pending')),
  provider_reference VARCHAR(120),    -- gateway transaction id, for reconciliation
  created_at         TIMESTAMPTZ NOT NULL DEFAULT now()
);
CREATE INDEX ix_payments_payer ON payments (payer_account_id, created_at DESC);
CREATE UNIQUE INDEX ux_payments_provider_reference ON payments (provider_reference)
  WHERE provider_reference IS NOT NULL;  -- guards against double-processing a webhook

CREATE TABLE payouts (
  id                 BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  teacher_id        BIGINT NOT NULL REFERENCES teachers(id) ON DELETE RESTRICT,
  amount_minor      BIGINT NOT NULL CHECK (amount_minor >= 0),
  currency_code     CHAR(3) NOT NULL DEFAULT 'NGN',

```

```

method_id    BIGINT NOT NULL REFERENCES payment_methods(id) ON DELETE RESTRICT,
schedule     VARCHAR(10) NOT NULL CHECK (schedule IN ('weekly', 'biweekly', 'monthly')),
period_start DATE NOT NULL,
period_end   DATE NOT NULL,
status       VARCHAR(12) NOT NULL DEFAULT 'processing'
              CHECK (status IN ('processing', 'paid', 'failed')),
CONSTRAINT chk_payout_period CHECK (period_end > period_start)
);
CREATE INDEX ix_payouts_teacher_period ON payouts (teacher_id, period_start DESC);

```

3.6 Feedback & progress

```

CREATE TABLE feedback (
  id                BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  session_id       BIGINT NOT NULL UNIQUE REFERENCES sessions(id) ON DELETE RESTRICT,
  teacher_id       BIGINT NOT NULL REFERENCES teachers(id) ON DELETE RESTRICT,
  learner_profile_id BIGINT NOT NULL REFERENCES learner_profiles(id) ON DELETE RESTRICT,
  session_notes    TEXT NOT NULL,
  progress_rating  SMALLINT NOT NULL CHECK (progress_rating BETWEEN 1 AND 5),
  next_steps      TEXT,
  submitted_at     TIMESTAMPTZ NOT NULL DEFAULT now()
);
CREATE INDEX ix_feedback_learner ON feedback (learner_profile_id, submitted_at DESC);

CREATE TABLE progress_reports (      -- materialized rollup, see section 6 for refresh strategy
  learner_profile_id BIGINT PRIMARY KEY REFERENCES learner_profiles(id) ON DELETE CASCADE,
  attendance_pct     NUMERIC(5,2) NOT NULL DEFAULT 0,
  goal_progress_pct  NUMERIC(5,2) NOT NULL DEFAULT 0,
  sessions_completed INTEGER NOT NULL DEFAULT 0,
  updated_at        TIMESTAMPTZ NOT NULL DEFAULT now()
);

CREATE TABLE ratings (
  id                BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  session_id       BIGINT NOT NULL REFERENCES sessions(id) ON DELETE RESTRICT,
  rated_by_account_id BIGINT NOT NULL REFERENCES accounts(id) ON DELETE RESTRICT,
  teacher_id       BIGINT NOT NULL REFERENCES teachers(id) ON DELETE RESTRICT,
  stars            SMALLINT NOT NULL CHECK (stars BETWEEN 1 AND 5),
  comment          TEXT,
  created_at       TIMESTAMPTZ NOT NULL DEFAULT now(),
  UNIQUE (session_id, rated_by_account_id)  -- one rating per rater per session
);
CREATE INDEX ix_ratings_teacher ON ratings (teacher_id, created_at DESC);

```

4. Indexing strategy — consolidated

Index	Definition	Serves
ux_accounts_email	accounts (email) WHERE deleted_at IS NULL	Login lookup; partial so soft-deleted rows don't block re-registration.
ix_learner_profiles_owner	learner_profiles (owner_account_id)	Parent dashboard: list of managed children + own profile.
ix_assessments_learner	assessments (learner_profile_id,	Fetch latest assessment for matching.

Index	Definition	Serves
	submitted_at DESC)	
ix_teachers_verification_status	teachers (verification_status)	Admin review queue for pending verifications.
ix_teacher_availability_teacher	teacher_availability (teacher_id, day_of_week)	Render a teacher's weekly calendar.
ix_matches_teacher_status_pending	matches (teacher_id) WHERE status='proposed'	Teacher's pending match-request inbox — small hot slice of a large table.
ix_sessions_learner_scheduled	sessions (learner_profile_id, scheduled_at DESC)	Parent/student session history and “upcoming” view.
ix_sessions_upcoming	sessions (scheduled_at) WHERE status='scheduled'	Background reminder/notification job scan.
ux_payments_provider_reference	payments (provider_reference) WHERE NOT NULL	Idempotency guard against duplicate webhook delivery.
ix_payments_payer	payments (payer_account_id, created_at DESC)	Billing history screen.
ix_feedback_learner	feedback (learner_profile_id, submitted_at DESC)	Progress/feedback timeline on the dashboard.
ix_ratings_teacher	ratings (teacher_id, created_at DESC)	Teacher profile review list.

5. Security & compliance

- Minor data (learner_profiles for profile_type='child', including date_of_birth) is personal data of a child under NDPR (Nigeria) and comparable regimes in expansion markets — access to it must be scoped by row-level security to the owning parent account and platform staff with a documented need, not broad application-role access.
- password_hash uses argon2id, never a reversible or fast hash. No password or raw payment credential is ever stored — payment_methods holds only method type, never card/account numbers, which stay with the payment processor (PCI scope stays off this database).
- provider_reference on payments is the reconciliation key against the payment processor's own ledger — this database is the system of record for bookings and status, not for card data.
- Suggested Postgres ROW LEVEL SECURITY policy shape: learner_profiles rows are visible to owner_account_id = current_account(), plus a separate staff role for verification/support workflows, enforced at the database layer rather than trusted entirely to the application.
- All PII columns (accounts.email, accounts.phone, learner_profiles.full_name, learner_profiles.date_of_birth) should be covered by encryption at rest (managed disk encryption at minimum; column-level encryption for date_of_birth is worth evaluating given its sensitivity for minors).
- Audit logging: every write to learner_profiles, payments, and verification_status on teachers should emit to an append-only audit_log table (actor account, action, before/after, timestamp) — not modeled in section 3 to keep the core schema readable, but required before production launch.

6. Scaling & operations

- Partition sessions and payments by RANGE on created_at (monthly) once volume justifies it — both are append-heavy, time-ordered, and queried mostly by recent range; partitioning keeps indexes small and makes archiving old partitions cheap.
- progress_reports and teachers.rating_avg are denormalized rollups, not sources of truth. Refresh them via a background job or a Postgres trigger on feedback/ratings insert — do not let application code write to them ad hoc from multiple code paths, or they will drift.
- Route dashboard/reporting reads (parent progress view, admin analytics) to a read replica; keep the primary reserved for the booking/payment write path, which is the latency-sensitive one.

- Use a migrations tool with reversible, versioned migrations (e.g. Flyway, Sqitch, or the ORM's native migration system) — no hand-run DDL against production. Every CREATE TABLE in section 3 is a first migration; every ALTER after launch is a numbered follow-on.
- Connection pooling (PgBouncer in transaction mode) is required before this schema sees production traffic — BIGINT identity + frequent short writes (session status transitions, payment webhooks) will exhaust direct connections quickly under load.

7. Open engineering decisions

- Single Postgres instance vs. service-per-domain database: this schema assumes one instance behind one application for v1. If payments is later split into its own service for compliance isolation, payments/payouts move out and become an event-consuming read model here instead of a direct FK.
- JSONB usage is currently limited to assessments.availability, where structure is genuinely variable. Resist the temptation to reach for JSONB elsewhere (e.g. teacher profile fields) just to avoid a migration — it defeats indexing and constraint enforcement.
- teacher_availability models weekly recurring slots only; if the product needs date-specific exceptions (holiday blackouts, one-off availability), that's an additive table, not a rework of this one.
- The chk_minor_requires_consent constraint (section 3.2) is flagged, not silently fixed, as an example of a rule that looks right in a CHECK but needs to move to a trigger or application-layer validation before this ships.